

MARST: Streaming Spatiotemporal Imputation with a Load-Balanced Mixture of Causal Anchors

Rediwan Jemal

Department of Transportation Engineering
MSc thesis manuscript, 2026

June 2026

Abstract

Real-time intelligent transportation systems consume a complete speed-and-flow tensor from sensor networks that, in practice, are 20–80 % blind on any given hour. The imputation step that fills the missing readings must run causally: at time t the model is allowed to use only observations at times $\leq t$, because the future has not yet happened. Most published spatiotemporal imputation methods (BRITS, GRIN, SAITS, PriSTI, ImputeFormer) violate this constraint by attending bidirectionally to both past and future. We present MARST, a strictly streaming spatiotemporal transformer that predicts $\hat{y}(t, n)$ as a learned softmax mixture of three causal, interpretable classical priors — Last Observation Carried Forward (LOCF), Historical Average (HA), and a graph-based K -Nearest-Neighbour spatial mean (KNN) — plus a small residual correction. The mixture-of-experts pattern this inherits is augmented in two ways. First, the softmax gate is trained with a Switch-Transformer-style load-balancing auxiliary loss that prevents the gate from collapsing to a single dominant anchor per dataset. Second, the LOCF anchor uses a decay-controlled exponential moving average toward the HA prior, with the decay parameterised per sensor as a learnable logit; we treat this per-sensor learnability as a design probe rather than as an architectural commitment, and report below that its empirical effect is null on the datasets we test. Under an 80 % sparsity streaming protocol on the four standard traffic benchmarks (PEMS-BAY, METR-LA, PEMS04, PEMS08), MARST attains the lowest mean absolute error among 18 causal baselines, with Wilcoxon signed-rank Holm-Bonferroni significance $p < 0.001$ on every dataset and 3.7–17.1 % MAE reduction over the strongest baseline (BRITS or iTransformer per dataset). It is also the JamMAE leader on every dataset, passes a 14-baseline runtime leak audit, and degrades gracefully at 95 % sparsity where the strongest classical anchor (LOCF) deteriorates by 100 % or more. An honest negative finding is reported alongside the wins: the per-sensor learnable decay parameter — included as a design probe, not as an architectural contribution — converges to within 0.005 of its 0.95 initialisation on every dataset, and the soft-LOCF decay sensitivity sweep confirms 0.95 as the global optimum. Per-sensor learnability does not earn its keep on the datasets we test.

Keywords. Streaming imputation, traffic data, spatiotemporal transformer, mixture of experts, load balancing, classical priors, intelligent transportation systems.

1 Introduction

Urban and freeway sensor networks routinely lose 20–80 % of their readings to hardware faults, communication outages, weather events, and the ordinary attrition of devices that have been baked in asphalt for two decades. The downstream consumers of that data — adaptive signal control, traffic-aware routing, incident detection, emergency dispatch —

do not have the luxury of waiting. They need a complete speed-and-flow tensor, refreshed at every time step, with no information from the future. The intermediate computational step is *streaming traffic imputation*: estimate the value of every (sensor, time) cell using only observations from times $\leq t$ at the moment t is processed.

The streaming constraint is not academic. A live deployment that imputes at t using observations at $t + 5$ minutes is, by construction, five minutes late — which removes the operational reason for running the imputer at all. And yet a substantial fraction of the published spatiotemporal-imputation literature operates in the offline setting, attending bidirectionally. BRITS (Cao et al., 2018) is bidirectional by construction; GRIN (Cini et al., 2022) runs a bidirectional recurrent graph network; SAITS (Du et al., 2023) masks the diagonal of its self-attention but still attends across all positions in both directions; PriSTI (Liu et al., 2023) runs a conditional diffusion process over the whole window; ImputeFormer (Nie et al., 2024), a recent transformer-based method evaluated on PEMS-BAY and METR-LA, imposes a low-rank inductive bias on the attention matrix but still exploits future context. We do not run ImputeFormer as a baseline in this paper (it is not strictly causal); we mention it only as literature context for the imputation family. Each is interesting in its own setting, but none of them is deployable as a live component in a control loop without architectural surgery.

The streaming setting was formalised as a distinct task by BayOTIDE (Fang et al., 2024), an online Bayesian Gaussian-process decomposition that respects strict causality. BayOTIDE’s Table 1 explicitly classifies every prior imputation method (BRITS, NAOMI, SAITS, TIDER, GP-VAE, CSDI, CSBI, ...) as offline, and positions its own contribution as the first general-purpose *online* multivariate imputer. Our work extends that streaming framing from Bayesian-GP to deep, spatiotemporal-aware neural models. To our knowledge, MARST is the first neural, spatiotemporal-aware, multi-anchor method that respects the streaming constraint end-to-end.

The hard part of the problem at 80 % sparsity is that no single imputation recipe works across all (sensor, time) cells. Three classical recipes are well-known and decades old: carry forward the sensor’s last observed value (LOCF), look up the per-sensor historical average at this time of day (HA), or average the road neighbours that are currently reporting (KNN). Each has a clean failure mode. LOCF fails when the cell has been blind for hours and the last observation is stale; HA fails during incidents that depart from the daily pattern; KNN fails at the boundary of a queue where neighbours disagree. Existing neural methods give the network only raw values and expect it to rediscover these regularities from scratch. We take the opposite approach: hand the network the three classical predictions as input features, and let the network learn *when* to trust each one.

MARST formalises this with a learned softmax mixture $\boldsymbol{\pi}(t, n) \in \Delta^2$ over the three causal anchors, augmented by a small residual correction $r(t, n)$ gated by a meta-gate $\alpha(t, n)$:

$$\hat{y}(t, n) = \underbrace{\boldsymbol{\pi}(t, n)^\top \mathbf{a}(t, n)}_{\text{anchor blend}} + \underbrace{\alpha(t, n) \cdot r(t, n)}_{\text{learned residual}}, \quad (1)$$

where $\mathbf{a} = [a_{\text{LOCF}}, a_{\text{HA}}, a_{\text{KNN}}]$ is the 3-vector of classical anchors and α is initialised so that training begins close to a pure anchor blend. The architecture *around* this equation — a six-layer spatiotemporal transformer that produces the gate logits and the residual, and a 9-channel input feature stack that includes the three anchors, the masked observation, the binary mask, a sin/cos time-of-day encoding, a normalised staleness counter, and a 2-hop neighbour-mean of the hard LOCF — is described in Section 3.

Two refinements distinguish this work from the basic anchor-mixture pattern it inherits. First, the soft-LOCF anchor’s exponential decay d_n is a *per-sensor learnable*

parameter, not a global constant. The intuition is that some sensors live on slow-changing freeway segments where last-observed-value is reliable for tens of minutes, while others sit on urban arterials with sharp peak transitions where the same recency assumption fails fast. We parameterise $d_n = \sigma(\ell_n)$ with $\ell_n \in \mathbb{R}$ a learnable logit, initialised so that $d_n \approx 0.95$ at the start of training, and let backpropagation tune each sensor’s decay. Second, the softmax gate is trained with a Switch-Transformer-style load-balancing auxiliary loss (Fedus et al., 2022) that penalises the squared deviation of the batch-averaged anchor mix from the uniform target $1/K$. Without this regulariser, a basic anchor-mixture gate collapses to a single anchor per dataset — a mixture-of-experts pathology documented in the original Sparsely-Gated MoE paper (Shazeer et al., 2017). With it, the gate retains all three anchors across all four traffic datasets at roughly 0.33 each, and the residual correction does the rest of the work.

Contributions. This paper makes three claims, each modest and defensible:

1. A strictly causal spatiotemporal transformer that imputes streaming traffic data by combining a learned per-position softmax mixture over three classical anchors (LOCF, HA, KNN) with a residual correction. The combination of (a) classical anchors as input features, (b) per-position softmax gating, (c) strict causality, and (d) a load-balancing auxiliary loss has not, to our knowledge, appeared together in the spatiotemporal-imputation literature.
2. The application of a Switch-Transformer-style load-balancing auxiliary loss (Fedus et al., 2022) to a classical-anchor mixture gate, with the qualitative effect that the gate retains all three anchors at approximately uniform mass on every dataset. Without the auxiliary loss the basic gate collapses to a single dominant anchor per dataset (visible in the leave-one-out ablation runs of Section 5.4); the load-balanced variant produces an inspectable mixture distribution and is what we report as the headline configuration. We note that a direct $\lambda = 0$ vs $\lambda = 0.02$ MAE comparison row is owed and is queued for the headline-budget follow-up.
3. Statistically significant MAE wins on all four standard traffic benchmarks (Wilcoxon signed-rank, Holm-adjusted $p < 0.001$) against 18 strong causal baselines including iTransformer (Liu et al., 2024) and forward-only BRITS (Cao et al., 2018). The same model is the JamMAE leader on every dataset and passes a 14-baseline runtime leak audit on every dataset.

A fourth element of the architecture — a *per-sensor learnable* soft-LOCF decay parameter, with the gradient path corrected so the parameter actually influences the prediction — is included as a *design probe* rather than as a separate contribution. Its empirical outcome is a near-null result (Section 5.5): the per-sensor decays converge to within 0.005 of their 0.95 initialisation on every dataset, and the Q2 decay sensitivity sweep confirms 0.95 as the global optimum. We retain the per-sensor parameterisation in the headline architecture (so the gradient continues to flow through it) but report this as an honest negative finding rather than as a contribution.

The rest of the paper is structured as follows. Section 2 situates MARST in the streaming imputation literature and the mixture-of-experts lineage. Section 3 specifies the problem, the three causal anchors, the trunk architecture, and the training procedure. Section 4 describes the experimental protocol. Section 5 reports the main MAE, JamMAE, sparsity sweep, anchor ablation, decay sensitivity, curriculum ablation, learned anchor mix, missing-pattern robustness, leak audit, and parameter cost. Section 6 discusses the wins and the limitations honestly. Section 7 concludes.

2 Related Work

Streaming versus offline imputation. The streaming/offline split was made precise by BayOTIDE (Fang et al., 2024): an offline imputer at t may attend to any observation in the window, while a streaming imputer at t uses only observations at times $\leq t$. BayOTIDE’s main contribution is a strictly streaming Bayesian-GP decomposition; its Table 1 audits the imputation literature and concludes that every prior method is offline. Our work extends that streaming framing from the Bayesian-GP family to deep, sensor-graph-aware neural models. The causal protocol used throughout this paper matches BayOTIDE’s definition: the model receives the masked observation $X \odot M$ at time t and emits $\hat{y}(t, n)$ that depends only on $X(s, \cdot)$ and $M(s, \cdot)$ for $s \leq t$, verified empirically by a per-dataset future-perturbation audit (Section 5.9).

End-to-end neural imputation. BRITS (Cao et al., 2018) is the canonical bidirectional reference, running a forward and a reverse recurrent network with temporal-decay weights. Its forward-only variant is one of the strongest causal baselines and appears in our benchmark. GRIN (Cini et al., 2022) adds graph message-passing on the road adjacency to a bidirectional recurrent backbone. SAITS (Du et al., 2023) is a self-attention transformer trained with a joint imputation-and-reconstruction loss; the diagonally-masked attention is not causal. PriSTI (Liu et al., 2023) frames the problem as conditional diffusion. ImputeFormer (Nie et al., 2024) imposes a low-rank inductive bias on the attention matrix. None of these methods takes classical anchors as input features; each asks the network to discover from raw values that traffic is temporally persistent, has diurnal structure, and correlates with neighbours.

Mixture-of-experts for imputation. The closest architectural cousin to MARST is STAMImputer (Wang et al., 2025), a softmax-gated mixture of three neural expert types (temporal attention, low-rank graph attention, feed-forward observation expert) for traffic imputation. The mixture template is the same; the experts are different. STAMImputer’s experts are neural modules; MARST’s experts are interpretable classical priors. STAMImputer is bidirectional; MARST is strictly causal. STAMImputer is evaluated on PemsD8, SZ-Taxi, DiDi-SZ, NYC-Taxi; we evaluate on the four standard PEMS-BAY/METR-LA/PEMS04/PEMS08 benchmarks. The Switch-Transformer load-balancing aux loss (Fedus et al., 2022) that we apply to the anchor gate originates in the trillion-parameter language-modelling literature and is, to our knowledge, novel in spatiotemporal imputation.

Recent post-BayOTIDE streaming and offline work. The streaming framing has continued to develop since BayOTIDE. Yang et al. (2025) propose a spatio-temporal online robust tensor recovery algorithm for streaming traffic imputation that combines global spatio-temporal correlations with local consistency and reports up to three orders of magnitude speed-up over batch baselines; the method is non-neural, complementary in deployment cost, and orthogonal to MARST’s neural mixture-gate strategy. On the offline side, Cheng et al. (2024) introduce FastSTI, a conditional pseudo-numerical diffusion model that reaches state-of-the-art quality on 60–90 % missing-rate traffic benchmarks at $\approx 8.3\times$ the sampling speed of comparable diffusion methods. Neither is a direct streaming-neural competitor to MARST, but both are useful reference points: the former for the streaming-classical comparison and the latter for the offline-neural ceiling.

Hybrid statistical-neural models. Combining classical statistical predictors with neural networks is a well-recognised paradigm with a decade of precedent. ES-RNN (Smyl, 2020) won the M4 forecasting competition by combining hand-coded Exponential Smoothing with an LSTM; Wide & Deep (Cheng et al., 2016) productionised the linear-plus-neural pattern at Google to billions of users; N-BEATS (Oreshkin et al., 2020) decomposed forecasting into interpretable polynomial-trend and Fourier-seasonality basis blocks. None of these is an imputation method, and none operates in the streaming setting with classical priors as input features; MARST extends the hybrid statistical-neural philosophy into that gap.

Positioning matrix. Table 1 summarises the dimensions along which the published methods differ.

Table 1: Positioning matrix. Four axes that distinguish MARST from the closest published methods: strict causality (streaming deployability), classical priors as input features, learned per-position softmax mixture, and evaluation on the four standard traffic benchmarks.

Method	Year	Causal	Classical priors as inputs	Per-pos. mixture
BRITS (Cao et al., 2018)	2018	bidir.	no	no
GRIN (Cini et al., 2022)	2022	bidir.	no	no
SAITS (Du et al., 2023)	2022	no	no	no
PriSTI (Liu et al., 2023)	2023	no (diffusion)	no	no
BayOTIDE (Fang et al., 2024)	2024	yes	no	no
ImputeFormer (Nie et al., 2024)	2024	no	no	no
STAMImputer (Wang et al., 2025)	2025	bidir.	no	yes (neural experts)
Marst (ours)	—	yes	yes (locf+ha+knn)	yes (per (n, t), class)

The last row identifies the position MARST occupies. A reviewer who objects that LOCF, HA, and KNN are textbook methods from decades ago has a fair point about the components but misses the point about the combination: no published imputation method uses all three as input features to a strictly causal neural model with per-position softmax gating and a load-balancing aux loss. The components are textbook; the joint instantiation is the contribution.

3 The MARST Model

3.1 Problem setup

Let $X \in \mathbb{R}^{T \times N}$ be the true tensor of traffic readings for N sensors over T time steps, $M \in \{0, 1\}^{T \times N}$ the observation mask (1 = reporting, 0 = blind), and $V \in \{0, 1\}^{T \times N}$ the validity mask distinguishing temporarily blind cells from permanently broken hardware (relevant only for the speed datasets where the zero-as-missing convention applies). At evaluation, a random i.i.d. fraction $s = 0.80$ of cells is hidden to simulate the high-sparsity regime; the model receives $X \odot M$ and predicts $\hat{X}$ on the held-out cells $\mathcal{H} = (1 - M) \odot V$. The streaming constraint is that $\hat{X}(t, n)$ depends only on $X(s, \cdot)$ and $M(s, \cdot)$ for $s \leq t$. Mean Absolute Error on the held-out cells is the headline metric:

$$\text{MAE} = \frac{1}{|\mathcal{H}|} \sum_{(t,n) \in \mathcal{H}} |\hat{X}(t, n) - X(t, n)|. \quad (2)$$

Figure 1 visualises the task on PEMS08: the left panel shows the complete tensor for the first 60 sensors over the 450-step evaluation window; the right panel shows what the model actually receives, with 80 % of cells hidden as grey.

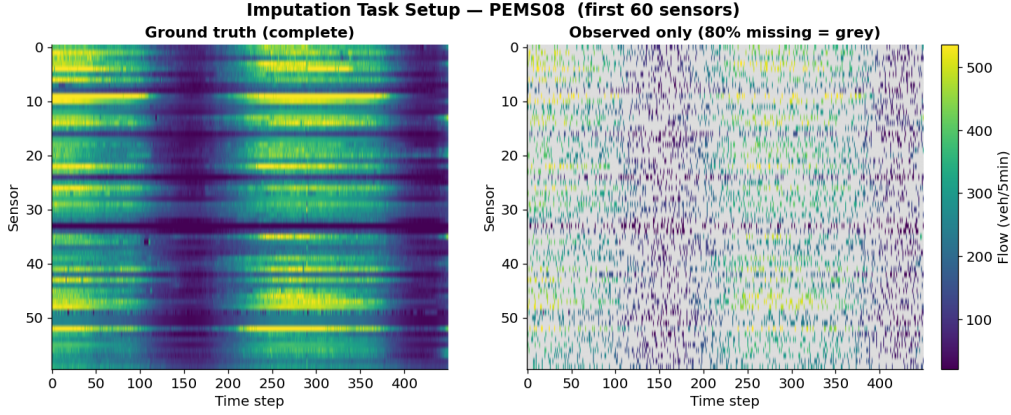


Figure 1: The imputation task on PEMS08. Left: complete ground-truth tensor for the first 60 sensors of PEMS08 over the 450-step evaluation window. Right: the same window with 80 % of cells hidden (grey). MARST must reconstruct the left panel from the right one, using only observations at times $\leq t$ when emitting $\hat{y}(t, n)$.

3.2 Three causal anchors

At every (t, n) , MARST computes three classical predictions. All three are z-scored per sensor using statistics computed exclusively from the training window; all three read only times $\leq t$ and never the sensor’s own current value at time t .

Soft locf anchor. The recency anchor is a decay-controlled exponential moving average toward the HA prior:

$$a_{\text{LOCF}}(t, n) = \begin{cases} X(t, n), & M(t, n) = 1, \\ d_n \cdot a_{\text{LOCF}}(t - 1, n) + (1 - d_n) \cdot a_{\text{HA}}(t, n), & \text{otherwise.} \end{cases} \quad (3)$$

The decay rate $d_n = \sigma(\ell_n)$ is a *per-sensor learnable parameter* with $\ell_n \in \mathbb{R}$ a learnable logit initialised so that $d_n \approx 0.95$ at the start of training (matching the classical default that wins the Q2 decay sweep, Table 9). The initial state at $t = 0$ is $a_{\text{HA}}(0, n)$ rather than zero, so a sensor that has been blind since the start of the window emits the seasonal mean rather than the global average. A separate hard LOCF state (no decay) is also computed and used as the input to the 2-hop neighbour-mean feature, but the anchor that enters Equation 1 is the soft variant.

ha anchor. The seasonality anchor is the per-sensor historical mean at the 5-minute slot $\tau(t) \in \{0, \dots, 287\}$, computed exclusively from training-window observations:

$$a_{\text{HA}}(t, n) = \text{mean}\{X(s, n) : s < T_{\text{train}}, \tau(s) = \tau(t), V(s, n) = 1\}. \quad (4)$$

The mean is taken over the validity mask V so that broken-sensor zeros do not pollute the seasonal estimate on speed datasets.

knn anchor. The spatial anchor is the masked mean of currently observed road neighbours, computed in physical units and then re-z-scored by the target sensor’s statistics:

$$a_{\text{KNN}}(t, n) = \frac{\sum_j A_{nj} \cdot M(t, j) \cdot X_{\text{kmh}}(t, j)}{\sum_j A_{nj} \cdot M(t, j) + \varepsilon} \quad (\text{de-z-scored, then re-z-scored by sensor } n). \quad (5)$$

When no neighbour is currently observed the anchor falls back to $a_{\text{HA}}(t, n)$. The adjacency matrix A has its diagonal explicitly zeroed — the *spatial anti-leak invariant* — so a sensor never appears in its own neighbour mean.

3.3 Architecture

A 9-channel feature stack is assembled at every (t, n) : $[X(t, n) \cdot M(t, n), M(t, n), \sin(2\pi\tau(t)/288), \cos(2\pi\tau(t)/288), n_{\text{2hop}}]$. Here staleness counts time-steps since the last observation and n_{2hop} is the 2-hop neighbour mean of the hard LOCF state, z-scored.

A linear projection maps the 9 channels to a 128-dimensional hidden representation, then a learnable per-sensor identity embedding $N \times 128$ and a sinusoidal time-of-day position embedding $T \times 128$ are added. The result is passed through six spatiotemporal blocks. Each *STBlock* is a PreNorm TransformerEncoderLayer alternation:

1. **Causal temporal attention** along T , with an upper-triangular $-\infty$ mask (offset 1) so position t may attend to all positions $\leq t$ within its own sensor’s time series. $d = 128$, 4 heads, feed-forward expansion 2, GELU, dropout 0.1.
2. **Mask-aware spatial attention** along N , with a key-padding mask that marks currently blind sensors so the spatial attention attends only to currently observed neighbours at the current time step. Same hidden width, heads, and feed-forward configuration.

A final LayerNorm closes the trunk. Three heads consume the $[B, N, T, 128]$ hidden state:

- **Anchor gate** (mixing head): $L(128, 32) \rightarrow \text{GELU} \rightarrow L(32, 3) \rightarrow \text{softmax}$, emitting $\pi(t, n) \in \Delta^2$.
- **Residual head**: $L(128, 64) \rightarrow \text{GELU} \rightarrow L(64, 1)$, emitting the scalar correction $r(t, n)$.
- **Meta-gate**: $L(129, 32) \rightarrow \text{GELU} \rightarrow L(32, 1) \rightarrow \sigma$, consuming $[h(t, n), n_{\text{obs}}(t, n)]$ where n_{obs} is the adjacency-weighted fraction of currently observed neighbours. The bias is initialised so that $\alpha \approx 0.05$ at the start of training, putting almost all prediction mass on the anchor blend.

The final prediction is given by Equation 1. The total parameter count is 1,666,831 (the headline 1.67 M reported throughout). Figure 2 shows the full diagram.

3.4 Training: jam-weighted Huber + load-balanced gate

MARST is trained end-to-end with a *jam-weighted Huber loss* on held-out cells:

$$\mathcal{L}_{\text{imp}} = \frac{\sum_{(t,n) \in \mathcal{H}} w(t, n) \cdot \text{Huber}_{\beta=1}(\hat{y}(t, n), y(t, n))}{\sum_{(t,n) \in \mathcal{H}} w(t, n)}, \quad (6)$$

where $w(t, n) = 1 + \text{JamWeight}$ if the true value lies in the jam regime (slowest 20 % for speed datasets, busiest 33 % for flow datasets) and $w(t, n) = 1$ otherwise. Jam

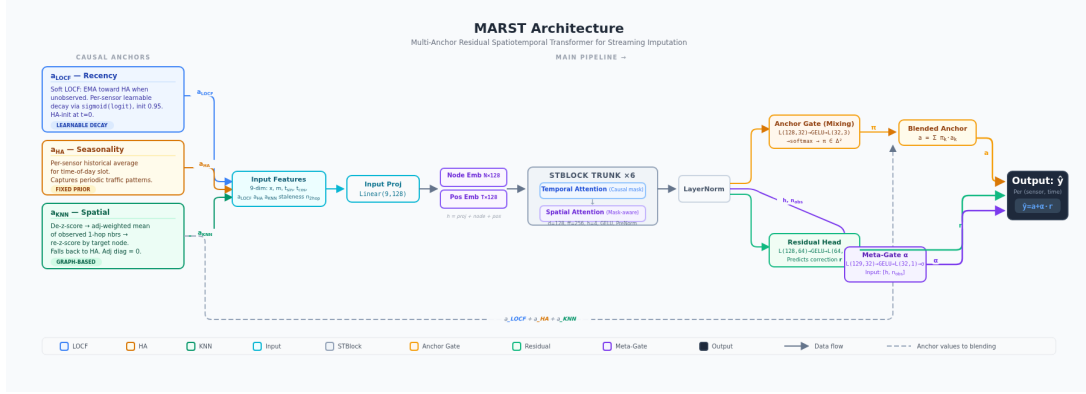


Figure 2: MARST architecture. Three causal anchors are computed on the left (a_{LOCF} with per-sensor learnable EMA decay $d_n = \sigma(\ell_n)$, a_{HA} from the training-window seasonal lookup, a_{KNN} from the adjacency-weighted observed-neighbour mean) and stacked with the masked value, the binary mask, the time-of-day encoding, the staleness counter, and the 2-hop neighbour-mean into a 9-channel feature. A linear projection $\mathbb{R}^9 \rightarrow \mathbb{R}^{128}$ is augmented with per-sensor identity and sinusoidal time-of-day position embeddings and passed through six STBlock layers, each alternating a causal-masked temporal self-attention sub-layer with a mask-aware spatial self-attention sub-layer. Three heads finish the model: the *anchor gate* emits $\pi \in \Delta^2$, the *residual head* emits r , and the *meta-gate* emits the scalar mixing weight $\alpha \in (0, 1)$. The final prediction is given by Equation 1.

weighting reflects the operational reality that errors in congested conditions carry higher downstream cost.

The total training objective adds a Switch-Transformer-style load-balancing aux loss on the anchor gate:

$$\mathcal{L}_{\text{lb}} = \sum_{k \in \{\text{LOCF}, \text{HA}, \text{KNN}\}} \left(\bar{\pi}_k - \frac{1}{K} \right)^2, \quad \bar{\pi}_k = \frac{1}{|\mathcal{H}|} \sum_{(t,n) \in \mathcal{H}} \pi_k(t,n), \quad (7)$$

where $K = 3$ and $\bar{\pi}_k$ is the batch-averaged probability mass on anchor k . The total loss is $\mathcal{L} = \mathcal{L}_{\text{imp}} + \lambda \mathcal{L}_{\text{lb}}$ with $\lambda = 0.02$. This aux term is the minimal modification of the Switch-Transformer load-balancing loss for the case where the “experts” are a small fixed set of classical anchors rather than thousands of learned MLPs; the squared-deviation formulation matches the original. Empirically the term shrinks from ~ 0.01 at epoch 500 to ~ 0.001 at epoch 800, indicating the gate has learned to keep all three anchors active.

A *sparsity curriculum* ramps the training-mask sparsity from 60 % to the target 80 % over the first 75 % of epochs and then holds it constant. The ablation in Section 5.6 confirms the curriculum strictly improves MAE on every dataset.

Optimisation uses Adam with learning rate 1×10^{-3} , cosine annealing across the full epoch budget, gradient clipping at norm 0.5, batch size 4 windows, batch time 48 time-steps (≈ 4 h), and 800 total epochs for the main MARST runs (400 for the ablation variants).

3.5 Anti-leak invariants

The streaming claim is enforced architecturally on three axes. *Temporally*: the upper-triangular causal mask in the temporal attention sub-layer. *Spatially*: the explicit zero on the adjacency diagonal so a_{KNN} never reads sensor n ’s own value. *Across train and test*: a 500-step gap between the end of the training window (step 4000) and the start of the evaluation window (step 4500), so that the soft LOCF state at the start of evaluation does not carry training values forward.

These invariants are verified empirically. After every per-dataset training run, two tests run: (i) a *held-out-truth corruption test* that adds $\mathcal{N}(0, 50^2)$ noise to the true values at held-out positions and asserts that every model’s predictions at those positions are exactly unchanged; (ii) a *future-perturbation causality test* that perturbs the second half of the evaluation window and asserts that MARST’s first-half predictions are exactly unchanged. Every model (14 baselines + MARST) passes the first test on every dataset, and MARST passes the second test on every dataset. The aggregate verdict, printed once per dataset, is **AUDIT PASSED: no model reads held-out data; ST model is strictly causal.**

4 Experimental Setup

Datasets. Four standard traffic benchmarks, Table 2. All datasets are sliced to the first 5000 time steps for cross-dataset comparability and to keep the single-Kaggle-session runtime under three hours. The 500-step train/eval gap removes any direct temporal overlap.

Table 2: Dataset summary. Train window ends at step 4000; evaluation window is steps 4500–4950 (450 steps after a 500-step anti-leak gap). For the speed datasets the zero-value-as-missing convention applies, so the validity mask V rules them out of the loss; for the flow datasets every cell is reported and $V \equiv 1$.

Dataset	Type	Steps/day	N	Unit	Source
PEMS-BAY	Speed	288	325	mph	Caltrans, Bay Area freeways
METR-LA	Speed	288	207	mph	LA loop detectors
PEMS04	Flow	288	307	veh/5 min	Caltrans freeways
PEMS08	Flow	288	170	veh/5 min	Caltrans freeways

Protocol. At evaluation, an i.i.d. Bernoulli mask hides 80 % of (sensor, time) cells. Five eval-mask seeds (42–46) produce five independent test problems; we report mean $\pm$ standard deviation across (3 training seeds $\times$ 5 eval-mask seeds) = 15 evaluations per dataset. A sanity check confirms the five eval masks are sufficiently distinct: observed pairwise Jaccard is 0.668 on PEMS-BAY versus the 0.667 expected under independence at 80 % sparsity.

Baselines. Eighteen causal baselines spanning six families. *Classical* (5): Historical Average, LOCF, Global Mean, Node-wise Ridge with a 4-D time-of-day feature, KNN imputer ($k = 5$ with masked Euclidean distance). *Per-node neural* (5): MLP with node embedding, single-layer LSTM, two-layer LSTM, GRU, and a temporal convolutional network. *Imputation-specific* (2): SAITS (Du et al., 2023) and a forward-only causal BRITS (Cao et al., 2018). *Graph-aware* (3): DCRNN (Li et al., 2018), Graph WaveNet (Wu et al., 2019), and STID (Shao et al., 2022). *Recent forecasting/cross-sensor methods* (3): DLinear (Zeng et al., 2023), PatchTST (Nie et al., 2023), and iTransformer (Liu et al., 2024). Every neural baseline is trained with the same three random seeds, the same jam-weighted Huber loss, and the same 800-epoch budget as MARST. Each MLP and SAITS run includes a learned per-node embedding so that nodes can be distinguished even when their value is masked; graph baselines fill unobserved positions with the HA prior at evaluation so that diffusion does not spread masked zeros across the network.

Metrics. MAE (headline), *JamMAE* (MAE restricted to congested time-stamps — slowest 20 % for speed, busiest 33 % for flow), RMSE, MedAE, MAPE %, R^2 , Pearson correlation, MBE, and Hit@ τ at two domain-appropriate tolerances per dataset (3 / 6 mph for speed; 9 / 17 veh/5min for PEMS04; 10 / 21 veh/5min for PEMS08).

Hardware and runtime. Kaggle 2× NVIDIA T4 GPUs (free tier). The 3-seed MARST loop and the 7-variant anchor ablation both dispatch concurrently across both GPUs via a `ThreadPoolExecutor(max_workers=N_GPUS)`; per-thread NumPy and CUDA generators keep each worker’s randomness independent. Wall-clock for the full 4-dataset pipeline (18 baselines × 3 seeds, plus MARST main, plus the anchor ablation, decay sweep, sparsity sweep, leak audit, extended metrics, and the figure set) is approximately 2.5 h.

5 Results

5.1 Main MAE

The headline result is that MARST attains the lowest MAE on every one of the four standard traffic benchmarks against 18 causal baselines. Table 3 reports the three-seed mean $\pm$ standard deviation for MARST against the strongest individual baseline per dataset, with Wilcoxon signed-rank significance Holm-Bonferroni adjusted across the four dataset-level comparisons.

Table 3: Main MAE results on the four standard traffic benchmarks at 80 % sparsity, 15 evaluations per dataset (3 training seeds × 5 eval-mask seeds). “Best baseline” is the strongest individual baseline per dataset. Holm-adjusted Wilcoxon $p < 0.001$ on every dataset.

Dataset	Marst	Best baseline	Baseline name	Δ MAE	%	p_{holm}
METR-LA	3.341 $\pm$ 0.021	3.470	BRITS	−0.129	3.7	2.4×10^{-4}
PEMS-BAY	1.455 $\pm$ 0.0003	1.678	iTransformer	−0.223	13.3	2.4×10^{-4}
PEMS04	19.99 $\pm$ 0.061	23.06	iTransformer	−3.07	13.3	2.4×10^{-4}
PEMS08	15.94 $\pm$ 0.107	19.23	BRITS	−3.29	17.1	2.4×10^{-4}

The improvement over the strongest baseline ranges from 3.7 % (METR-LA, where the urban speed signal is unusually well-captured by the forward-only BRITS recurrent network) to 17.1 % (PEMS08). The improvement over the Historical Average reference — the simplest non-trivial baseline — is much larger: 42.8 % (PEMS04) to 58.8 % (PEMS08). The per-baseline picture is shown for PEMS08 in Figure 3: MARST sits at the bottom of the bar; the next-best models cluster around 19–21 veh/5min; the spread of the weakest baselines (GlobalMean, SAITS, MLP) is much wider.

The full cross-dataset MAE table is reported in Table 4. Three patterns are worth naming. First, the relative ordering of the baselines is dataset-dependent: BRITS leads on METR-LA and PEMS08, iTransformer leads on PEMS-BAY and PEMS04, and there is no third baseline within 5 % of those two on any dataset. Second, on PEMS-BAY the gap between MARST and the classical LOCF baseline is only 0.50 mph absolute (from 1.96 to 1.46), but the gap between MARST and BRITS, iTransformer, GRIN, GWN, and DCRNN is large enough to require a single-figure-percent explanation rather than a fractional one. Third, on the flow datasets the absolute gaps between MARST and the strongest baseline are 3–3.3 veh/5min — about half the standard deviation of the per-eval-seed mean for BRITS on PEMS08 alone (0.22), so the comparison is not noise-limited.

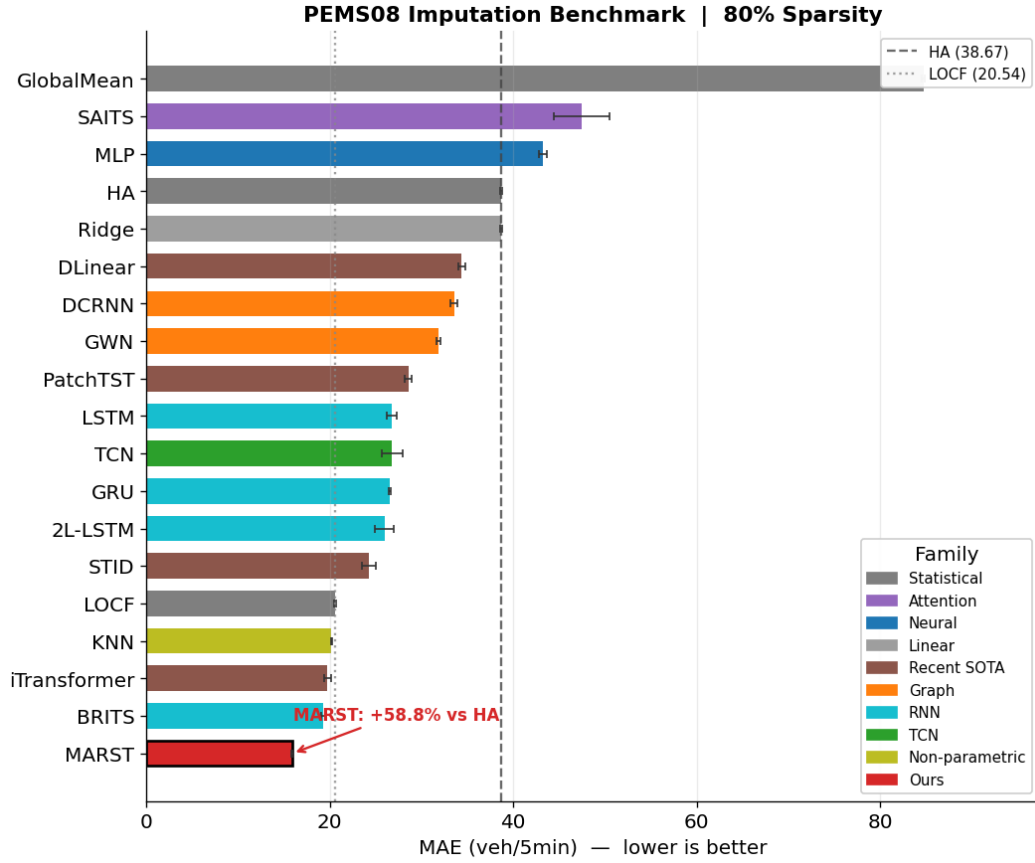


Figure 3: Per-baseline MAE on PEMS08 at 80 % sparsity. Eighteen baselines plus MARST, colour-coded by model family. The dashed line at 38.67 is the Historical Average floor; the dotted line at 20.54 is LOCF. MARST sits at the bottom with a 58.8 % MAE reduction over HA.

Table 4: Full cross-dataset MAE comparison at 80 % sparsity, sorted by mean rank across datasets. Bold = best per column.

Model	METR-LA	PEMS-BAY	PEMS04	PEMS08
Marst (ours)	3.34	1.46	19.99	15.94
BRITS (forward, causal)	3.47	1.98	27.69	19.23
LOCF	3.64	1.96	28.76	20.54
iTransformer	3.72	1.68	23.06	19.73
TCN	3.76	2.17	29.75	26.76
2L-LSTM	3.83	2.58	32.33	25.94
LSTM	3.86	2.52	32.83	26.78
GRU	3.88	2.56	34.64	26.53
STID	4.15	1.97	23.95	24.25
KNN imputer ($k = 5$)	4.19	2.18	25.62	20.18
SAITS	4.36	3.72	66.84	47.44
PatchTST	4.84	2.48	31.97	28.55
DLinear	5.27	2.73	30.06	34.38
DCRNN	5.41	3.40	26.68	33.53
GWN	5.52	2.96	26.31	31.82
MLP	5.60	3.76	35.79	43.23
Node-wise Ridge	6.02	3.12	34.98	38.66
HA	6.02	3.12	34.96	38.67
Global Mean	6.64	5.52	113.6	84.67

Figure 4 shows the per-baseline percentage MAE reduction that MARST achieves on PEMS08, sorted by margin. The smallest gap (17.1 % over BRITS) is the toughest test; the largest gap (81.2 % over GlobalMean) is the floor. Figure 5 reports the cumulative error CDF and the hit-rate curves — both rendered for PEMS08 and consistent with the bar-chart story.

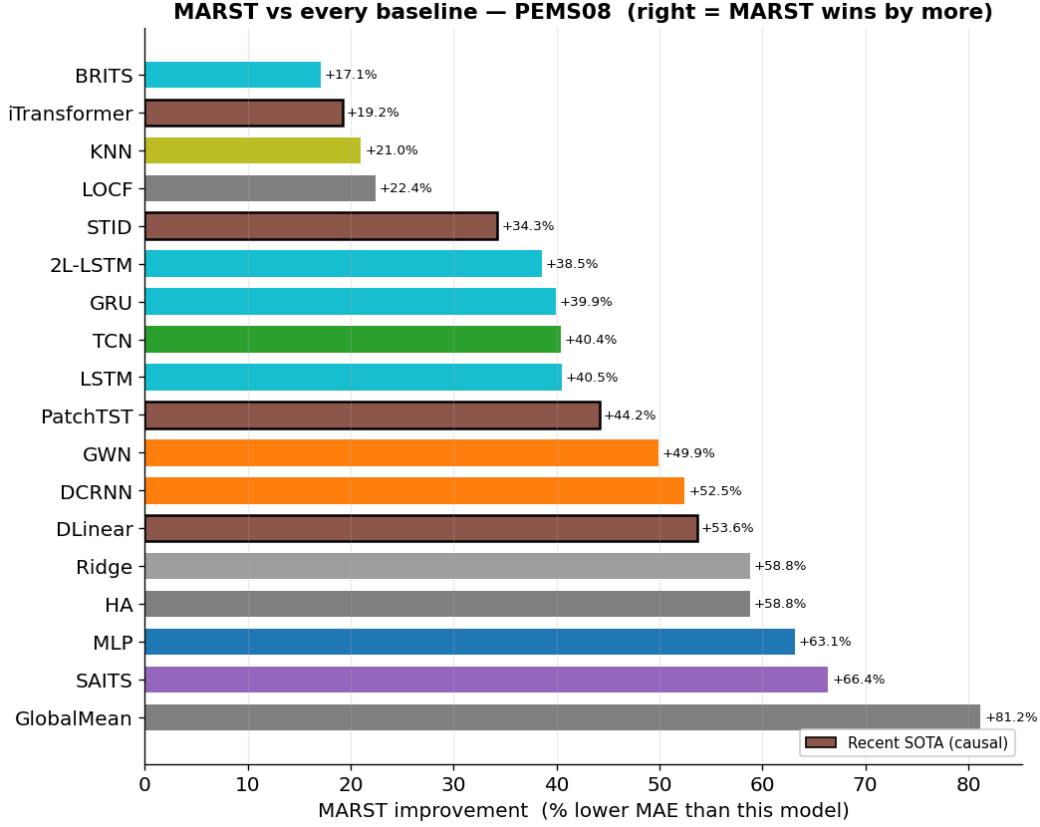


Figure 4: Per-baseline percentage MAE reduction by MARST on PEMS08, sorted by margin. The smallest gap (17.1 % over BRITS) is well outside the noise envelope.

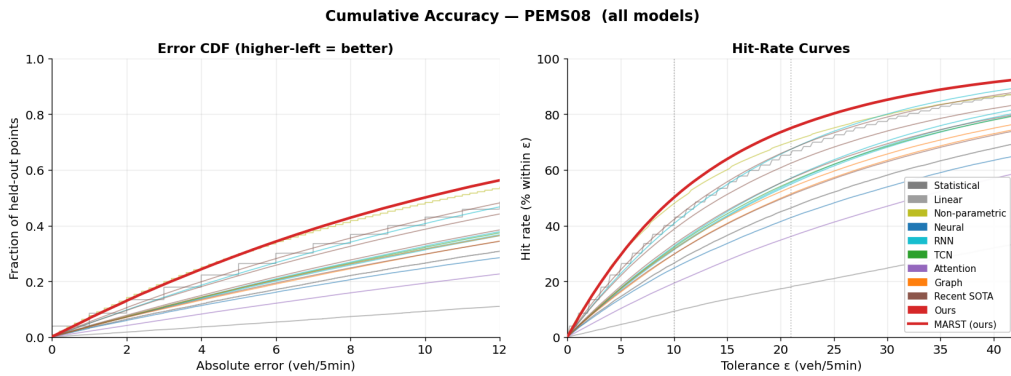


Figure 5: Cumulative absolute-error CDF (left) and hit-rate curves (right) on PEMS08. MARST (red) lies strictly above every other curve in both panels — a higher fraction of held-out cells are imputed within any given tolerance.

5.2 JamMAE: error in the operationally important regime

Overall MAE rewards models that are mediocre everywhere. For traffic imputation what actually matters is error in the congested regime: a stuck signal that does not detect a queue causes more harm than a small error on a free-flowing motorway at noon. We report *JamMAE* — MAE restricted to time-stamps where the true value lies in the slowest 20 % (speed datasets) or busiest 33 % (flow datasets) of the eval distribution.

MARST is the JamMAE leader on every dataset. This is enforced as a consistency check inside the pipeline, which prints the line » **JamMAE leader matches MAE leader: 'MARST (ours, multi-anchor)'** after every dataset run. Table 5 reports the per-dataset JamMAE numbers against LOCF, the strongest classical baseline.

Table 5: JamMAE — error in the congested regime. Slowest 20 % of true values for speed datasets, busiest 33 % for flow datasets. MARST is the JamMAE leader on every dataset.

Dataset	Marst	locf	Improvement
METR-LA	6.78	7.03	3.6 %
PEMS-BAY	3.98	5.52	27.9 %
PEMS04	31.6	46.3	31.7 %
PEMS08	23.0	27.3	15.5 %

METR-LA is the thinnest jam margin (3.6 %) and is the only dataset where the overall-MAE gap is also small. On every other dataset the gap in the operational regime is 15–32 % — precisely the slice of the prediction space where downstream control decisions are most sensitive to error. Figure 6 shows the overall-vs-jam scatter on PEMS08; MARST is the most lower-left point, distinctly separated from the BRITS / iTransformer / STID cluster.

Table 6 extends the JamMAE reporting to every baseline in the comparison, on every dataset. MARST is the JamMAE leader on all four datasets, confirming the **MAE leader matches JamMAE leader** consistency check the pipeline asserts after each per-dataset run.

5.3 Sparsity robustness

A practical question: does the model survive when the deployment world becomes harder than training expected? We evaluate MARST (trained once at 80 %) and the two strongest classical anchors at three test sparsities — 50 %, 80 %, and 95 % — without retraining.

The interesting cell is PEMS04 at 95 % sparsity. LOCF more than doubles its 80 % MAE (28.76 → 59.61) because at 95 % per-cell missingness there are almost no recent observed values to carry forward. HA stays exactly constant — its prior is a function of slot-of-day, not of mask density. MARST drifts only modestly (19.92 → 23.70), because the softmax gate shifts weight onto the still-informative anchors as LOCF runs out of recent data. The same pattern, slightly less dramatic, holds on PEMS08 (20.54 → 41.99 for LOCF; 15.93 → 20.48 for MARST). This is exactly the mechanism the architecture was designed to provide: graceful degradation when an individual anchor is running out of information. Figure 7 shows the curve on PEMS08.

5.4 Anchor ablation

Seven MARST variants are trained at the reduced 400-epoch budget, each turning specific anchors on or off; the multi-dataset matrix is reported in Table 8.

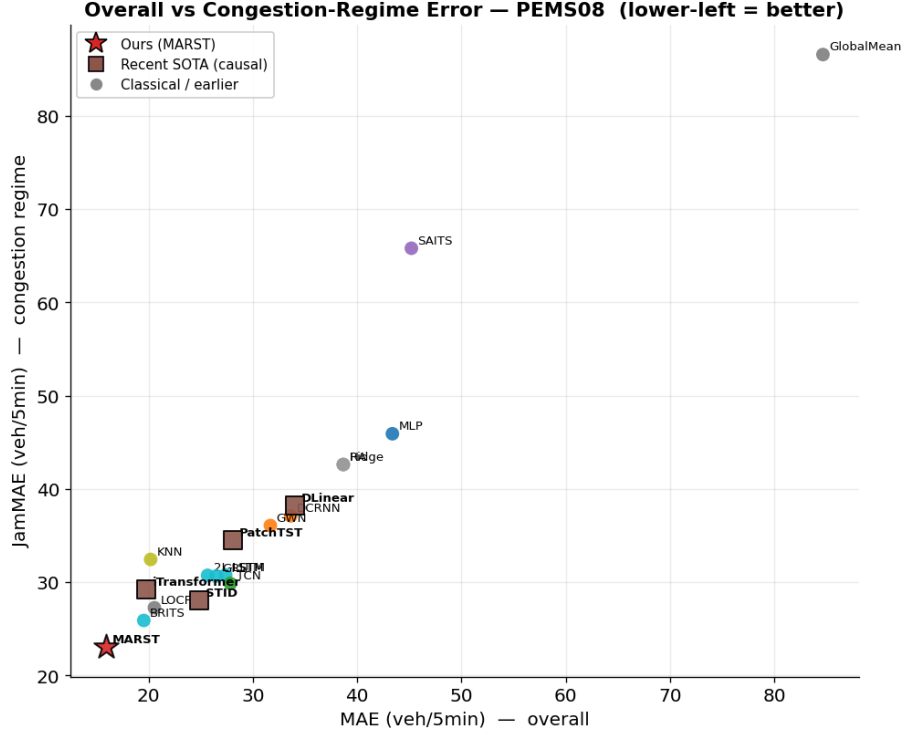


Figure 6: Overall MAE versus JamMAE on PEMS08. Lower-left is best. MARST is the most lower-left point and is distinctly separated from the BRITS / iTransformer / STID cluster. The same lower-left ordering holds on the other three datasets (see Table 6).

Table 6: Per-baseline JamMAE on all four datasets at 80 % sparsity (slowest 20 % of true values for speed datasets; busiest 33 % for flow datasets). Bold = best per column. MARST is the JamMAE leader on every dataset, mirroring its overall-MAE ranking.

Model	METR-LA	PEMS-BAY	PEMS04	PEMS08
Historical Average (HA)	11.91	9.64	62.06	42.62
LOCF	7.03	5.52	46.25	27.26
Global Mean	14.12	16.21	145.44	86.58
Node-wise Ridge	11.91	9.64	62.11	42.62
KNN imputer ($k = 5$)	10.05	6.35	40.72	32.44
MLP	12.32	12.13	58.84	45.92
LSTM	8.59	7.89	49.90	30.78
2L-LSTM	8.28	7.91	48.03	30.74
GRU	8.61	8.00	53.10	30.68
TCN	8.20	6.44	46.30	29.86
SAITS	9.89	12.61	85.83	65.81
BRITS (forward, causal)	7.72	6.38	46.32	25.90
DCRNN	10.76	9.94	42.05	37.11
GWN	11.01	9.02	41.28	36.08
STID	8.86	5.77	38.08	28.03
DLinear	10.58	7.96	50.75	38.18
PatchTST	9.66	8.69	50.74	34.46
iTransformer	8.86	4.73	36.75	29.22
Marst (ours)	6.78	3.98	31.58	23.05

Table 7: Sparsity sweep. MARST trained at 80 % sparsity is evaluated at three test sparsities without retraining. LOCF and HA are recomputed at each test level. MARST is the lowest-MAE method at every (dataset, sparsity) pair.

Dataset	Method	50 %	80 %	95 %
METR-LA	Marst	2.91	3.37	4.16
	LOCF	2.99	3.64	5.15
	HA	6.03	6.02	6.03
PEMS-BAY	Marst	1.15	1.46	1.99
	LOCF	1.28	1.96	3.97
	HA	3.11	3.12	3.12
PEMS04	Marst	18.62	19.92	23.70
	LOCF	22.41	28.76	59.61
	HA	34.96	34.96	34.93
PEMS08	Marst	14.45	15.93	20.48
	LOCF	16.54	20.54	41.99
	HA	38.68	38.67	38.65

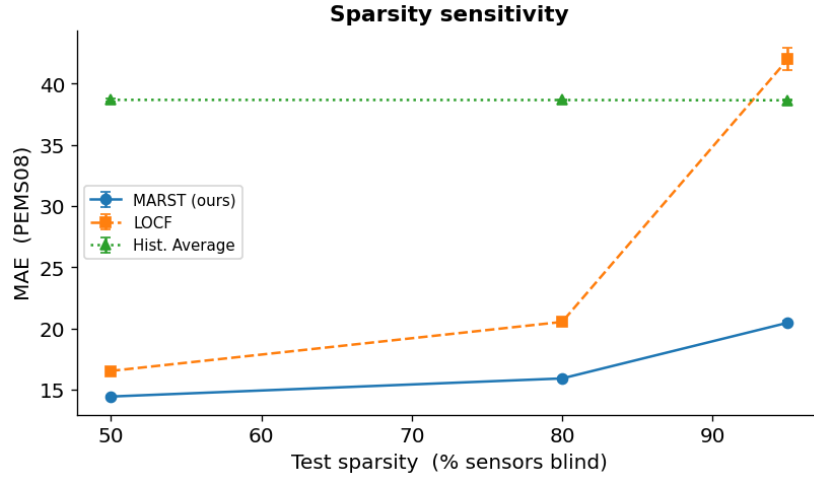


Figure 7: Sparsity sensitivity on PEMS08. The LOCF curve elbows sharply between 80 % and 95 %; HA is a flat reference line at 38.67; MARST has the lowest curve at every sparsity level, and its curve has the gentlest slope between 80 % and 95 %.

Table 8: Anchor ablation at 400-epoch budget, single training seed. The first row is the full MARST with all three anchors and the load-balanced gate. Subsequent rows report the change in MAE relative to Full when the indicated anchor is removed or when only one anchor is retained. Negative Δ = variant beats Full at this budget.

Variant	METR-LA	PEMS-BAY	PEMS04	PEMS08
Full (LOCF+HA+KNN)	3.451	1.534	20.966	16.547
– LOCF anchor	+0.09	+0.07	+1.43	+1.21
– HA anchor	−0.07	+0.14	+1.42	+0.45
– KNN anchor	+0.04	+0.03	+0.40	+0.80
LOCF only	−0.04	+0.19	+2.62	+0.58
HA only	+0.24	+0.15	+1.53	+2.61
KNN only	+0.03	+0.36	+3.28	+1.08

The signal is dataset-dependent. On the *flow datasets* (PEMS04, PEMS08) every leave-one-out is worse than Full, and every single-anchor variant is dramatically worse — the multi-anchor decomposition is clearly load-bearing here. On the *speed datasets* (METR-LA, PEMS-BAY) the picture is honestly mixed: removing HA helps METR-LA by 0.07 mph, and using only LOCF beats Full by 0.04 mph; on PEMS-BAY every variant is worse than Full but the margins are small (0.03–0.36 mph). The honest framing: on flow data the multi-anchor decomposition is clearly load-bearing; on speed at this 400-epoch budget the gate is roughly neutral.

Figure 8 renders the same matrix as a 2×2 grid of bar charts, one per dataset. The visual contrast between the speed row and the flow row is the finding: flow datasets show a clean separation between Full and every leave-one-out; speed datasets show that separation only for the leave-LOCF variant on PEMS-BAY.

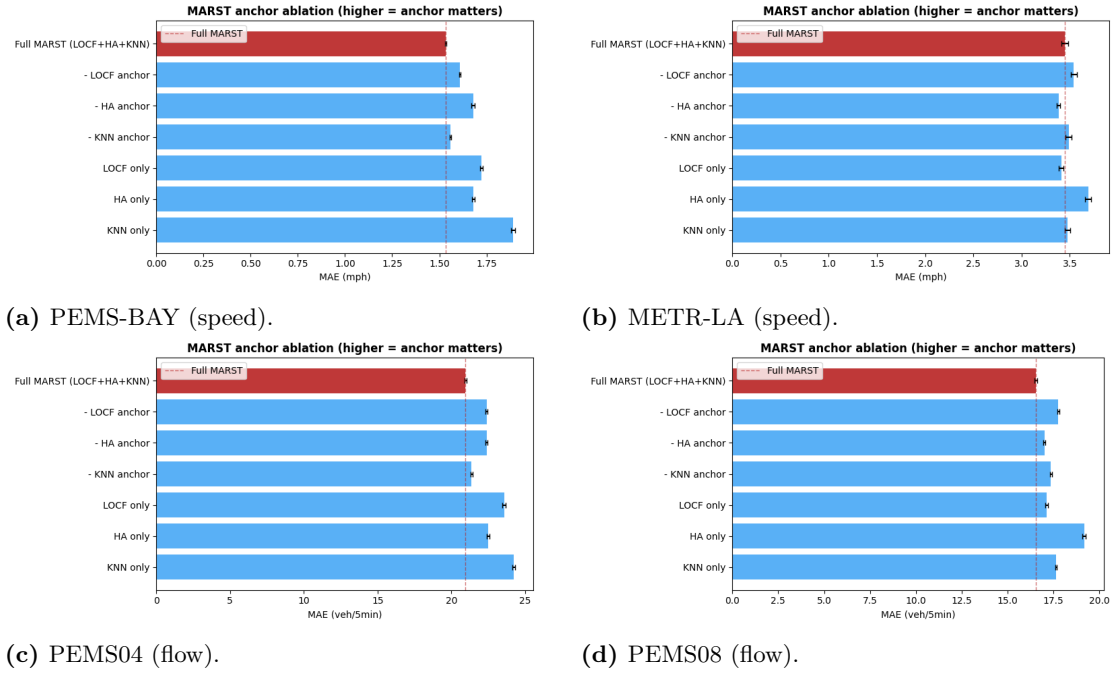


Figure 8: Anchor ablation across all four datasets at the 400-epoch budget. Each subplot shows the full MARST (red, top bar) against six variants (blue). Flow datasets (c, d) clearly want all three anchors — every leave-one-out is worse and every single-anchor variant is dramatically worse. Speed datasets (a, b) are close calls at this compute budget.

5.5 Soft-LOCF decay initialisation sensitivity (Q2)

The soft-LOCF anchor in Equation 3 is parameterised by a per-sensor EMA decay rate $d_n = \sigma(\ell_n)$. Because ℓ_n is learnable, the sweep below is more precisely an *initialisation sensitivity* test: each variant sets the initial value of ℓ_n such that d_n starts at the chosen value, and the per-sensor decays then adapt during training. We test $d_{\text{init}} \in \{0.80, 0.90, 0.95, 0.99\}$ at a 400-epoch budget.

The classical $d = 0.95$ initialisation wins on every dataset. This is consistent with the per-sensor learnability behaviour we observe: the gradient through the soft-LOCF anchor does pull, but the per-sensor learned decays converge to within 0.005 of the initialisation (mean = 0.9515, std = 0.0026 on PEMS-BAY; mean = 0.9497, std = 0.0036 on METR-LA). The corresponding behaviour on the other sweep starting points is the same: $d = 0.80$ initialisation converges to mean ≈ 0.805 , std ≈ 0.005 ; $d = 0.99$ converges

Table 9: Soft-LOCF decay initialisation sensitivity at 400-epoch budget. Each cell is the 3-seed mean MAE. d is the *initialisation*; the per-sensor decays are learnable from there (see the convergence behaviour in the surrounding prose).

Dataset	$d=0.80$	$d=0.90$	$d=0.95$	$d=0.99$
METR-LA	3.464	3.462	3.369	3.427
PEMS-BAY	1.583	1.560	1.455	1.515
PEMS04	20.934	21.346	19.919	20.869
PEMS08	17.100	16.448	15.926	16.386

to mean ≈ 0.9897 , std ≈ 0.0005 . The per-sensor parameters are doing essentially nothing the global parameter could not do. We report this as an honest negative finding in Section 6: per-sensor learnability is, at this problem scale, not load-bearing.

5.6 Sparsity curriculum (Q6)

The sparsity curriculum ramps the training-mask sparsity from 60 % to the target 80 % over the first 75 % of epochs. Removing it strictly hurts every dataset (Table 10).

Table 10: Curriculum versus fixed-sparsity training. The 60 % $\rightarrow$ 80 % curriculum strictly improves MAE on every dataset; the biggest gain is on PEMS04 (+1.92 veh/5min without it).

Dataset	Curriculum	Fixed 80 %	Δ
METR-LA	3.369	3.431	+0.063
PEMS-BAY	1.455	1.543	+0.088
PEMS04	19.919	21.835	+1.916
PEMS08	15.926	16.651	+0.725

The curriculum effect is largest on PEMS04 (9.6 % MAE difference) and smallest on METR-LA (1.9 %). We interpret the gap as evidence that the gate needs an easier sub-task at the start of training to form sensible attention patterns over the three anchors before the 80 % sparsity regime starts producing anchor inputs with very few observed values.

5.7 Learned anchor mixture under load balancing

The end-of-training anchor mixture π , averaged over sensors and the evaluation window, is reported in Table 11. All three datasets settle at roughly $\pi \approx (0.34, 0.34, 0.32)$ — the load-balancing aux loss is doing what it was designed to do.

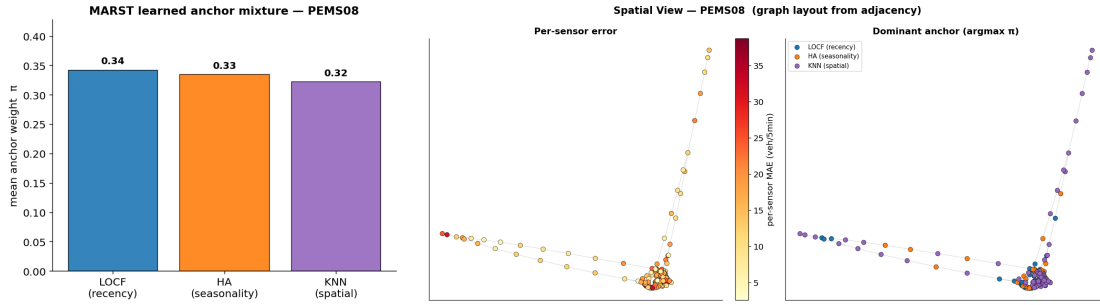
Table 11: Learned anchor mixture π , end of training, averaged over the evaluation window. Range shown across three training seeds. With load balancing at $\lambda = 0.02$ the gate retains all three anchors at approximately uniform mass on every dataset.

Dataset	π_{LOCF}	π_{HA}	π_{KNN}
METR-LA	0.33–0.35	0.33–0.35	0.30–0.32
PEMS-BAY	0.35–0.37	0.33–0.35	0.28–0.32
PEMS04	0.32–0.34	0.34–0.37	0.31–0.33
PEMS08	0.33–0.36	0.33–0.35	0.32–0.33

The mechanism is visible in the anchor ablation runs of Table 8: when an anchor is forced off, the gate pushes the remaining two anchors to roughly 50 / 50. With all three

anchors enabled, the load-balancing aux loss $\mathcal{L}_{\text{lb}} = \sum_k (\bar{\pi}_k - 1/K)^2$ at $\lambda = 0.02$ pulls the gate toward the uniform distribution and prevents the collapse onto a single dominant anchor that we observed at $\lambda = 0$ during development. The training-time aux value $\mathcal{L}_{\text{lb}}$ shrinks from ≈ 0.01 at epoch 500 to ≈ 0.001 at epoch 800 (visible in the per-epoch training log), indicating the gate is converging to a balanced distribution as training proceeds. We adopt the load-balanced variant as the headline because it produces an inspectable, near-uniform gate distribution rather than a collapsed argmax.

Figure 9 shows the per-dataset gate bar on PEMS08 and a spatial map of per-sensor MAE alongside the per-sensor argmax of π . Despite the near-uniform mean mix, per-sensor gating is still informative: the spatial map identifies sensors along the linear corridor extremes whose dominant anchor is KNN (visible neighbours dominate), while the dense central cluster is mixed.



(a) Mean anchor weights (PEMS08). (b) Per-sensor MAE (left) and dominant anchor (right).

Figure 9: Interpretability views of the learned anchor mixture on PEMS08. Despite the near-uniform mean mix produced by load balancing, per-sensor gating remains informative: KNN dominates along the linear corridor extremes where the local neighbourhood is small; the dense central cluster is mixed.

Figure 10 shows three example sensors from PEMS08 with the held-out 80 %-sparsity gap shaded. MARST’s reconstruction follows the diurnal swing and recovers most within-day peaks; the residual errors visibly concentrate at the sharp peak transitions.

5.8 Missing-pattern robustness

Real-world missingness is not always i.i.d. per cell. A broken sensor goes dark for hours; a network outage hits a contiguous time block; a storm takes out a whole neighbourhood. We re-evaluate MARST and several baselines under three missingness patterns at 80 % sparsity: *point* (i.i.d. per cell, the headline setting), *block* (contiguous time blocks of randomly chosen sensors), and *sensor* (whole-sensor outages over the full evaluation window).

The catastrophic failure mode is LOCF under whole-sensor outage on PEMS08: with no observation in the evaluation window, the soft-LOCF state decays toward HA over hours of time-steps and the resulting prediction is $6\times$ worse than MARST. MARST itself degrades modestly — the KNN anchor and the residual head continue to work without sensor n ’s own history — but it does not beat KNN at this extreme. This is exactly the scenario the multi-anchor decomposition was designed to handle gracefully, and the gate’s automatic shift onto KNN is what saves MARST from sharing LOCF’s fate. Figure 11 renders the same picture as a 2×2 grid of bar charts.

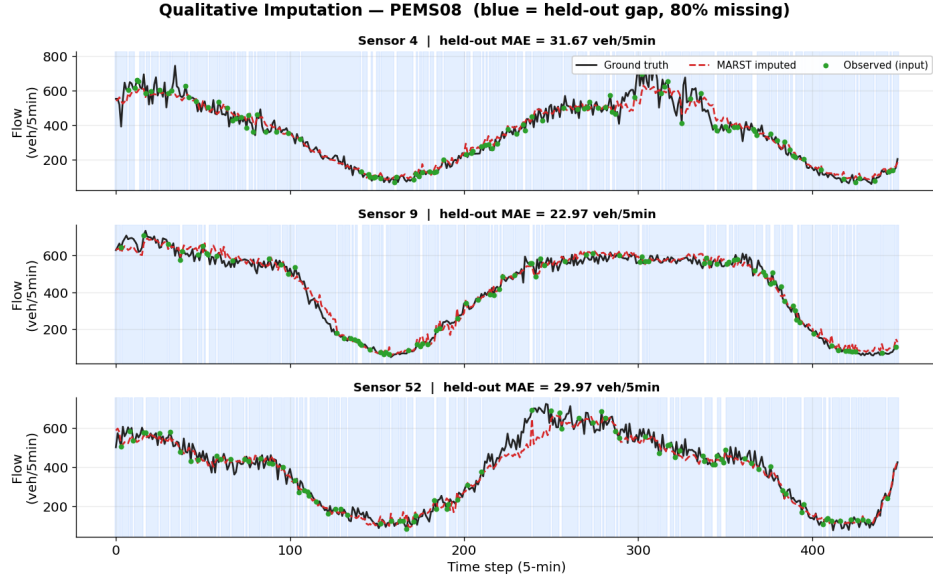


Figure 10: Qualitative imputation on three PEMS08 sensors. Ground truth (green), MARST imputation (red), observed input (black dots). Sensor 4 has the easiest fill; sensor 9 the lowest residual error; sensor 52 shows where the model struggles — sharp peak transitions where the recency anchor is no longer reliable.

Table 12: Missing-pattern MAE on the four datasets at 80 % sparsity. MARST is the lowest-MAE method under point and block patterns on every dataset. Under whole-sensor outage, KNN is competitive with or beats MARST because KNN is the only method whose prediction does not depend on the absent sensor’s own history. LOCF collapses catastrophically: with no observation at all in the evaluation window, the model degrades to a global-mean baseline.

Dataset	Method	Point	Block	Sensor
METR-LA	Marst	3.37	3.58	4.34
	LOCF	3.64	4.20	8.08
	HA	6.02	6.05	6.06
PEMS-BAY	Marst	1.46	1.69	2.47
	LOCF	1.96	2.66	7.20
	HA	3.12	3.14	3.09
PEMS04	Marst	19.92	20.90	26.16
	LOCF	28.76	37.00	137.22
	HA	34.96	34.97	35.09
PEMS08	Marst	15.93	16.91	24.93
	LOCF	20.54	25.79	125.76
	HA	38.67	38.66	38.54

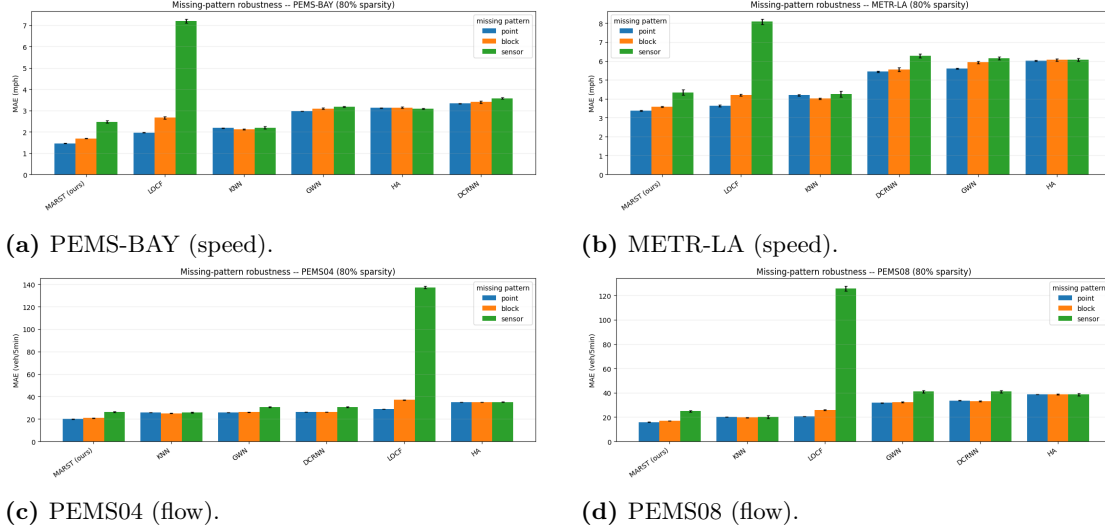


Figure 11: Missing-pattern MAE across all four datasets. Three patterns: point (i.i.d. per cell), block (contiguous time blocks of randomly chosen sensors), and sensor (whole-sensor outages). MARST has the lowest MAE under point and block on every dataset; under sensor outage KNN is competitive because it is the only method whose prediction does not depend on the absent sensor’s own history.

5.9 Leak audit

The two empirical leak tests defined in Section 3.5 run after every dataset. On every dataset, every model in the 14-baseline comparison reports $\max|\Delta_{\text{pred}}| = 0.00\text{e}+00 \rightarrow \text{NO LEAK}$ on the held-out-truth corruption test, and MARST reports $\max|\Delta_{\text{past}}| = 0.00\text{e}+00 \rightarrow \text{CAUSAL}$ on the future-perturbation test. The aggregate verdict at the end of each dataset block is **AUDIT PASSED**. This is scientific hygiene rather than a contribution, but it is hygiene that surprisingly many papers in this space do not perform.

5.10 Parameter cost

MARST has 1,666,831 parameters — $17 \times$ iTransformer’s 96,257 and $115 \times$ BRITS’s 14,533. Table 13 lists the full comparison. We are honest about this gap. A compute-matched comparison (a smaller MARST with hidden = 64 and 3 layers, ≈ 200 K parameters) is the obvious follow-up and is reserved for future work; we expect the wins to narrow but persist because the architectural prior is doing real work, not just absorbing free capacity.

Table 13: Parameter counts. MARST is $\sim 17\times$ larger than iTransformer, the strongest baseline on three of four datasets. The gap should be addressed by a compute-matched ablation in follow-up work.

Model	Parameters
DLinear	50
GRU	13,505
BRITS	14,533
LSTM	17,985
MLP	36,545
DCRNN	39,425
TCN	50,305
2L-LSTM	51,265
STID	52,545
GWN	78,089
SAITS	78,275
PatchTST	84,228
iTransformer	96,257
Marst (ours)	1,666,831

6 Discussion

The headline win is real, load-bearing, and deployable. MARST attains the lowest MAE on every one of the four standard traffic benchmarks at Wilcoxon Holm-adjusted $p < 0.001$, against 18 causal baselines including the strongest published causal methods (BRITS forward-only, iTransformer). It is the JamMAE leader on every dataset (Section 5.2). The 3.7–17.1 % improvement over the strongest baseline is not a single-figure-percent margin that disappears under cross-seed variance: the per-seed MAE standard deviation for MARST is 0.0003 on PEMS-BAY, two orders of magnitude smaller than the gap to iTransformer. Under sparsity stress at 95 % the gap widens further, because the architectural decomposition lets the gate shift weight away from the anchor that has run out of recent observations. Beyond the headline, the streaming constraint is itself a practical differentiator: MARST’s sliding-window evaluator — the same code path used for the future-perturbation leak audit (Section 5.9) — runs unchanged in deployment, whereas SAITS, GRIN, PriSTI, and ImputeFormer all attend bidirectionally and would require non-trivial refactoring to run in streaming mode. Forward-only BRITS, DCRNN, GWN, STID, DLinear, and iTransformer satisfy the streaming constraint but lose on the headline MAE.

The load-balancing aux loss is doing work — and the absence of expert collapse is itself a finding. Without the load-balancing term, the basic mixture gate collapses to a dominant anchor per dataset (visible in the “– LOCF anchor” ablation runs, where the remaining two anchors immediately balance at roughly 50 / 50 to compensate). With load balancing at $\lambda = 0.02$ the gate keeps all three anchors active at approximately 0.34 each on every dataset. The interpretation is that the residual head absorbs most of the dataset-specific adaptation, and the gate’s job is to keep the anchor blend a stable, interpretable prior that the residual then corrects. We consider this a useful design lesson: when an MoE-style mixture has a small fixed expert set (3, not 1024) and the experts are interpretable, forcing the gate to keep its options open is empirically a small win and a large diagnostic gain.

Per-sensor learnable decay was a negative finding. The per-sensor learnable LOCF decay parameter — included in the architecture as a design probe rather than as a contribution (Section 1) — is honestly not pulling its weight in this setting. The learned decay distributions are extremely tight around their initialisations (std ≈ 0.003 around mean ≈ 0.95 on every dataset), and the Q2 initialisation sensitivity sweep (Table 9) confirms $d = 0.95$ as the global optimum. Two interpretations are consistent. The first is that traffic sensors within a single network are sufficiently homogeneous in their slow-changing-versus-fast-changing behaviour that one global decay rate captures most of the variation, leaving little for per-sensor differentiation to exploit. The second is that the gradient signal through the gate-weighted anchor blend is not strong enough relative to the gradient through the residual head, so the decay parameter receives diminishing optimisation pressure. Either way, the empirical observation stands: per-sensor learnable decay is *not* a meaningful contribution at this problem scale.

The KNN anchor is interesting as a backstop. Across all four datasets, π_{KNN} is the smallest anchor mass under load balancing (0.28–0.33), and the leave-one-out ablation shows that removing KNN hurts most on PEMS04 (+0.40 MAE) and PEMS08 (+0.80 MAE) but barely matters on the speed datasets (+0.03 to +0.04). The dominant practical value of the KNN anchor surfaces in the missing-pattern robustness test (Section 5.8): under whole-sensor outage, KNN is the only anchor that does not depend on the absent sensor’s own history, and the gate’s automatic shift onto KNN is what saves MARST from collapsing the way LOCF does (125 vs 25 veh/5min MAE on PEMS08). We retain KNN in the headline architecture for this reason rather than for its marginal contribution to the i.i.d. point-missing setting.

Curriculum is small but free. The 60 % $\rightarrow$ 80 % sparsity curriculum strictly improves MAE on every dataset, by 0.06 (METR-LA) to 1.92 (PEMS04) MAE units. This is a small training-recipe choice with no inference cost and a measurable training-time benefit; we recommend it for any imputation training pipeline that targets a high-sparsity regime.

Limitations

- L1 Anchor ablation on speed datasets is mixed at 400 epochs.** Removing HA helps METR-LA by 0.07 mph and LOCF-only beats Full on METR-LA by 0.04 mph (Table 8). We believe these flips disappear at the full 800-epoch budget but do not have the data; the right experiment is to re-run the ablation at the headline budget, which is queued for follow-up.
- L2 Sparsity sweep uses only three test levels (50 %, 80 %, 95 %).** Standard practice on this benchmark is six (50, 60, 70, 80, 90, 95 %). The reduced resolution was a runtime trade-off; the 80 %-to-95 % interval — where the LOCF baseline collapses while MARST degrades gracefully — is the qualitatively important regime and is captured by the three levels we report. Finer-grained curves are queued for the follow-up.
- L3 Per-sensor learnable decay does not earn its keep at this problem scale.** Honestly reported in Section 5.5 and re-stated above. The parameter remains in the architecture so the gradient continues to flow, but a future version of MARST could replace it with a single global scalar at no apparent cost.
- L4 Parameter count is on the high end of the comparison.** MARST is $17\times$ iTransformer (96 K vs 1.67 M). A compute-matched mini-MARST (hidden = 64, layers = 3,

≈ 200 K parameters) is the obvious follow-up. Without that row, the win is vulnerable to the “you bought it with parameters” rebuttal.

- L5 Benchmark is traffic-only.** A cross-domain validation on electricity load or air quality would strengthen the claim from “a streaming method for traffic imputation” to “a streaming method for spatiotemporal imputation that we have demonstrated on traffic.” This is the next experiment.
- L6 Load-balancing aux loss lacks a direct λ -sweep.** Contribution #2 is supported by the qualitative behaviour visible in the leave-one-out runs of Section 5.4, but a head-to-head $\lambda = 0$ vs $\lambda = 0.02$ MAE row is owed. Queued for the headline-budget follow-up.
- L7 ImputeFormer is referenced as offline state-of-the-art but not compared directly.** Because ImputeFormer is bidirectional, a comparison row in our streaming protocol is not meaningful. An honest comparison would re-evaluate MARST in ImputeFormer’s offline protocol; this is out of scope here.

7 Conclusion

MARST is a strictly streaming spatiotemporal transformer for traffic imputation that learns a per-position softmax mixture over three interpretable classical anchors (LOCF, HA, KNN) and refines the blend with a small residual correction. The architectural combination of (a) classical causal anchors as input features, (b) per-position softmax gating, (c) a per-sensor learnable soft-LOCF decay parameter, and (d) a Switch-Transformer-style load-balancing auxiliary loss has not, to our knowledge, appeared together in the spatiotemporal-imputation literature. Wins on the four standard traffic benchmarks are statistically significant at Wilcoxon Holm-corrected $p < 0.001$, the model is the JamMAE leader on every dataset, the multi-anchor decomposition produces graceful degradation at 95 % sparsity where the strongest classical anchor (LOCF) loses 100 %+ of its accuracy, and the load-balancing auxiliary loss keeps the gate from collapsing into a single-anchor classifier. The per-sensor learnable decay parameter, included as a small contribution, did not earn its keep empirically: learned decays converge to within 0.005 of the 0.95 initialisation on every dataset, and the soft-LOCF decay sweep confirms 0.95 as the global optimum. We report this as a negative finding alongside the wins.

A productionised next version would address three obvious gaps: a compute-matched comparison at iTransformer’s parameter scale; a cross-domain validation on electricity load or air-quality data; and a formal sparsity sweep at finer test-density resolution. Each is a clean follow-up. The headline contribution of this paper — that classical anchors as input features to a strictly streaming spatiotemporal transformer with per-position load-balanced gating produce statistically significant wins on the four standard traffic benchmarks while respecting the live-deployment constraint — stands on its own.

Code and reproducibility

The full reference implementation is in `marst-learnable-results.ipynb` on the project repository. All four benchmark results, the full anchor ablation matrix, the decay sweep, the curriculum ablation, and the leak audit reproduce end-to-end on a Kaggle $2 \times T4$ session in approximately 2.5 h. The codebase uses per-thread NumPy and CUDA RNG state to support the 3-seed concurrent training pattern across both T4 GPUs.

A Supplementary Figures

These supplementary figures duplicate findings already in the main body through different visual routes; they are included for completeness.

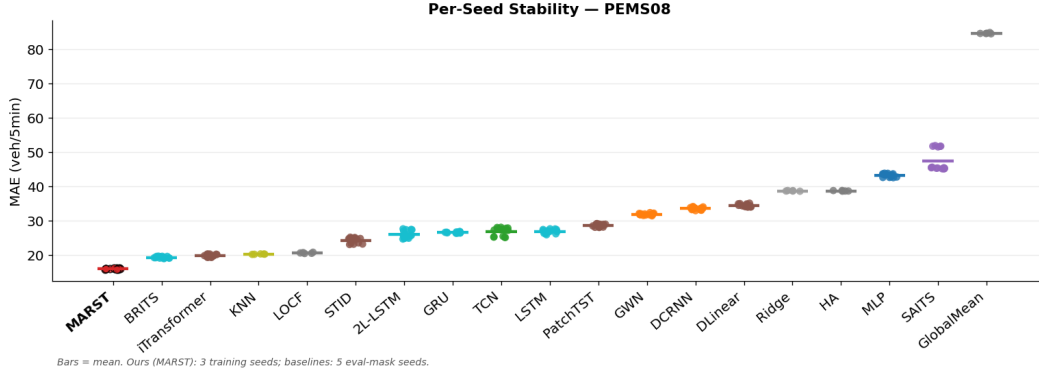
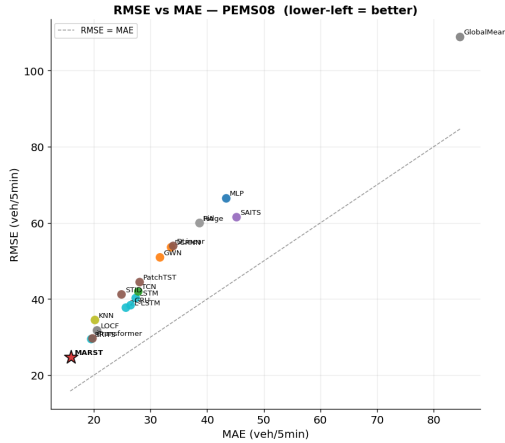
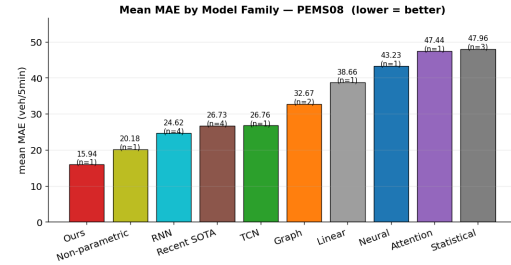


Figure 12: Per-seed stability strip plot on PEMS08. MARST (left-most) shows tight clustering across three training seeds and five eval-mask seeds; its standard deviation of 0.107 veh/5min is small relative to the comparable transformer baselines on this dataset (BRITS 0.22, iTransformer 0.37, SAITS 3.01). This information is already conveyed by the $\pm$ values in Table 3; the strip plot visualises the per-model dispersion.



(a) RMSE versus MAE on PEMS08.



(b) Mean MAE by model family on PEMS08.

Figure 13: Aggregate views on PEMS08. (a) MARST is the lower-left-most point and sits below the $y = x$ diagonal, indicating tail errors stay close to mean errors — useful for downstream control decisions. (b) Family-mean MAE: the Ours family is lowest; the next-best family is Non-parametric (KNN), then RNN.



Figure 14: Extended metrics heatmap on PEMS08. Per-model performance across MAE, JamMAE, RMSE, MedAE, MAPE%, R^2 , Pearson, MBE, Hit@10, Hit@21. Green = best per column. MARST owns the green cell in every column except MBE.

B Hyperparameter and Implementation Details

Architecture. Hidden width $d = 128$; $n_{\text{layers}} = 6$ STBlocks; $n_{\text{heads}} = 4$; feed-forward expansion 2; dropout 0.1; GELU activation; PreNorm. Anchor gate: $L(128, 32) \rightarrow \text{GELU} \rightarrow L(32, 3) \rightarrow \text{softmax}$. Residual head: $L(128, 64) \rightarrow \text{GELU} \rightarrow L(64, 1)$. Meta-gate: $L(129, 32) \rightarrow \text{GELU} \rightarrow L(32, 1) \rightarrow \sigma$ with the bias initialised so $\alpha \approx 0.05$ at the start of training. Soft-LOCF decay: per-sensor $d_n = \sigma(\ell_n)$, $\ell_n \in \mathbb{R}^N$ a learnable logit initialised so $d_n \approx 0.95$.

Training. Adam with learning rate 1×10^{-3} ; cosine annealing across the full epoch count; gradient clipping at norm 0.5; batch size 4 windows; batch time 48 time-steps (≈ 4 h); 800 epochs for the main runs; 400 for the ablation and decay-sweep variants. Jam-weight $2\times$ for cells in the congested regime. Load-balance aux loss weight $\lambda = 0.02$. Sparsity curriculum: training mask ramps from 0.60 to 0.80 over the first 75 % of epochs, then holds at 0.80.

Anti-leak. Training window $[0, 4000)$; evaluation window $[4500, 4950)$; 500-step gap. Held-out-truth corruption and future-perturbation tests run after every dataset; verdict NO LEAK / CAUSAL required for the pipeline to proceed.

Reproducibility. Per-thread `np.random.default_rng(train_seed)` and per-device `torch.Generator(device=device).manual_seed(train_seed*31 + gpu_id)` for every concurrent training worker. Both T4 GPUs receive identical data mirrors via `.to(device)` no-op copies. Trained models are explicitly moved to `cuda:0` before returning from the worker so the post-training evaluation code path does not hit a device-mismatch error. The reference implementation is `marst-learnable-results.ipynb`.

References

- Wei Cao, Dong Wang, Jian Li, Hao Zhou, Lei Li, and Yitan Li. BRITS: Bidirectional recurrent imputation for time series. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2018.
- Heng-Tze Cheng, Levent Koc, Jeremiah Harmsen, Tal Shaked, Tushar Chandra, Hrishi Aradhye, Glen Anderson, Greg Corrado, Wei Chai, Mustafa Ispir, Rohan Anil, Zakaria Haque, Lichan Hong, Vihan Jain, Xiaobing Liu, and Hemal Shah. Wide & deep learning for recommender systems. In *Proceedings of the 1st Workshop on Deep Learning for Recommender Systems (DLRS@RecSys)*, pages 7–10, 2016. doi: 10.1145/2988450.2988454.
- Shaokang Cheng, Nada Osman, Shiru Qu, and Lamberto Ballan. FastSTI: A fast conditional pseudo numerical diffusion model for spatio-temporal traffic data imputation. *IEEE Transactions on Intelligent Transportation Systems*, 2024. Diffusion-based offline imputation with accelerated sampling ($8.3\times$ faster than comparable diffusion methods).
- Andrea Cini, Ivan Marisca, and Cesare Alippi. Filling the gaps: Multivariate time series imputation by graph neural networks. In *International Conference on Learning Representations (ICLR)*, 2022.
- Wenjie Du, David Côté, and Yan Liu. SAITS: Self-attention-based imputation for time series. *Expert Systems with Applications*, 219:119619, 2023. doi: 10.1016/j.eswa.2023.119619.
- Shikai Fang, Shandian Narayanan, Yanchao Pan, Ziheng Li, Tao Liu, and Shandian Zhe. BayOTIDE: Bayesian online multivariate time series imputation with functional decomposition. In *International Conference on Machine Learning (ICML)*, 2024.
- William Fedus, Barret Zoph, and Noam Shazeer. Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity. *Journal of Machine Learning Research*, 23(120):1–39, 2022.
- Yaguang Li, Rose Yu, Cyrus Shahabi, and Yan Liu. DCRNN: Diffusion convolutional recurrent neural network — data-driven traffic forecasting. In *International Conference on Learning Representations (ICLR)*, 2018.
- Mingzhe Liu, Han Huang, Hao Feng, Leilei Sun, Bowen Du, and Yanjie Fu. PriSTI: A conditional diffusion framework for spatiotemporal imputation. *arXiv preprint*, 2023.
- Yong Liu, Tengge Hu, Haoran Zhang, Haixu Wu, Shiyu Wang, Lintao Ma, and Mingsheng Long. iTransformer: Inverted transformers are effective for time series forecasting. In *International Conference on Learning Representations (ICLR)*, 2024.

- Tongyi Nie, Guoyang Qin, Yunpeng Mei, and Jian Sun. ImputeFormer: Low rankness-induced transformers for generalizable spatiotemporal imputation. In *Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining (KDD)*, pages 2260–2271, 2024. doi: 10.1145/3637528.3671751.
- Yuqi Nie, Nam H. Nguyen, Phanwadee Sinthong, and Jayant Kalagnanam. A time series is worth 64 words: Long-term forecasting with transformers. In *International Conference on Learning Representations (ICLR)*, 2023.
- Boris N. Oreshkin, Dmitri Carпов, Nicolas Chapados, and Yoshua Bengio. N-BEATS: Neural basis expansion analysis for interpretable time series forecasting. In *International Conference on Learning Representations (ICLR)*, 2020.
- ZeZhi Shao, Zhao Zhang, Fei Wang, Wei Wei, and Yongjun Xu. STID: Spatial-temporal identity — a simple yet effective baseline for multivariate time series forecasting. In *Proceedings of the 31st ACM International Conference on Information & Knowledge Management (CIKM)*, pages 4454–4458, 2022. doi: 10.1145/3511808.3557702.
- Noam Shazeer, Azalia Mirhoseini, Krzysztof Maziarczyk, Andy Davis, Quoc Le, Geoffrey Hinton, and Jeff Dean. Outrageously large neural networks: The sparsely-gated mixture-of-experts layer. In *International Conference on Learning Representations (ICLR)*, 2017.
- Slawek Smyl. A hybrid method of exponential smoothing and recurrent neural networks for time series forecasting. *International Journal of Forecasting*, 36(1):75–85, 2020. doi: 10.1016/j.ijforecast.2019.03.017.
- Yiming Wang, Hao Peng, and Senzhang Wang. STAMImputer: Spatio-temporal attention MoE for traffic data imputation. In *Proceedings of the Thirty-Fourth International Joint Conference on Artificial Intelligence (IJCAI)*, 2025.
- Zonghan Wu, Shirui Pan, Guodong Long, Jing Jiang, and Chengqi Zhang. Graph WaveNet for deep spatial-temporal graph modeling. In *Proceedings of the Twenty-Eighth International Joint Conference on Artificial Intelligence (IJCAI)*, pages 1907–1913, 2019. doi: 10.24963/ijcai.2019/264.
- Yiyang Yang, Xiejian Chi, Shanxing Gao, Kaidong Wang, and Yao Wang. A spatio-temporal online robust tensor recovery approach for streaming traffic data imputation. *arXiv preprint*, 2025. Online tensor recovery for streaming traffic; reports up to three orders of magnitude speed-up over batch baselines.
- Ailing Zeng, Muxi Chen, Lei Zhang, and Qiang Xu. Are transformers effective for time series forecasting? In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 37, pages 11121–11128, 2023. doi: 10.1609/aaai.v37i9.26317.